Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks

Alvaro Rivera

2026-05-15

Abstract

Production software systems routinely include layers such as caches, object-relational mappers, message queues, distributed locks, application server frameworks, and orchestration clusters. These layers are treated as standard components of non-trivial deployments, yet they are rarely examined under a common explanation for why they exist.

This paper argues that many of these layers do not arise from structural requirements of the problem being solved, but from structural deficiencies of the underlying persistence model. It names this property *infrastructural symptom*. A layer exhibits infrastructural symptom when it compensates for a defect of the persistence model and loses justification when that defect is removed. Two diagnostic indicators — compensation and dissolution — give shape to this distinction, and an auditor workflow guides their application to any (layer, use-case) pair without requiring adoption of a specific alternative.

The paper presents a journaled actor-native system as an existence proof in which the canonical symptoms dissolve. Four mechanisms — locality, journal density, journal-as-causal-substrate, and self-contained substrate — account for the disappearance of seven common categories of compensating infrastructure. The construct extends through a ladder of additional layers to a limit case in which whole categories of business software become deliverable as a single appliance, as an architectural permission rather than an operational demonstration.

The contribution is analytic: a diagnostic typology that makes visible which parts of a stack are tied to the persistence model rather than to the problem domain.

Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks

TL;DR

Many layers taken for granted in production systems — caches, ORMs, message queues, distributed locks, application servers, orchestration clusters — do not exist because the problem requires them, but because the underlying persistence model does.

This paper names that property *infrastructural symptom*, defines the two diagnostic indicators that distinguish it from a structural requirement, provides an auditor workflow to read any layer under those indicators, and shows a journaled actor-native system in which the canonical symptoms disappear.

The construct is diagnostic, not prescriptive: it makes visible which parts of a stack compensate for the model rather than serve the problem.

Claims this paper makes

1. Some infrastructural layers in production deployments exist to compensate for structural deficiencies of the underlying persistence model rather than for structural requirements of the problem being solved (§3).
2. The property naming this distinction — *infrastructural symptom* — is read through two diagnostic indicators: compensation (the layer compensates for an identifiable model defect) and dissolution (the layer loses justification when the defect is removed) (§3).
3. An auditor workflow guides the application of the two indicators and produces a reading — model-dependent, structural, or unclassified — per (layer, use-case) pair (§5).
4. A journaled actor-native model instantiates a system in which four mechanisms — locality, journal density, journal-as-causal-substrate, self-contained substrate — dissolve the seven canonical categories of compensating infrastructure (§4.1).
5. The Redis case admits the workflow with per-use-case reading: six use-cases dissolve as symptoms; two remain structural (§4.2).
6. The ladder of compensated layers is open-ended; the workflow extends to backup tooling, service mesh, distributed-tracing/APM, and future emergent layers (§6).
7. The model permits appliance-class deployment as a limit case in which categories of business software become physically shippable — an architectural permission, not an operational proof (§7).

1. Introduction

This paper makes an analytic contribution: a diagnostic typology for reading existing stacks. It identifies and names a structural pattern that prior literature has documented case by case without articulating as one (the typology); derives the indicators under which the pattern reads as model-dependent (the criteria); and presents an instantiation in which those indicators read positively and the pattern is absent (the existence proof). The genre is closer to *theory for analyzing* in the sense of Gregor (2006) than to design theory: evidence is presented in the form of a working artifact, but the artifact illustrates the lens's reading rather than validating it. The contribution is conceptual; the instantiation confirms that the pattern is contingent rather than structurally necessary, not that any specific system is the only way to dissolve it.

Production software systems are routinely deployed alongside layers of supporting infrastructure: in-memory caches, object-relational mappers, message queues, distributed locks, application server frameworks, orchestration clusters, full operating systems. These layers are treated as standard components of any non-trivial deployment. Architectural discussions assume their presence by default; tutorials and reference architectures position them as foundational; engineering teams budget for them, staff them, and monitor them. Their absence is treated as a property of toy systems.

This paper names a property that some — not all — of these layers share. We call this property *infrastructural symptom*: a layer is an infrastructural symptom when it exists to compensate for a structural deficiency in the underlying persistence model rather than for a structural requirement of the problem being solved. The distinction is routinely elided; the two cases are treated alike in architecture, in tooling, and in engineering culture. Once the property is named, each layer in a given stack becomes auditable under the question: is this compensating for something, and if so, for what?

Section 2 situates the paper among prior work. Section 3 defines the construct and derives the two diagnostic indicators that distinguish infrastructural symptom from structural requirement. Section 4 presents the instantiation: a journaled actor-native system in which the canonical layers — caches, glue queues, locks, application server scaffolding — are absent not by omission but by structural irrelevance. Section 5 specifies the auditor workflow for reading a stack under the typology. Section 6 generalizes from the canonical case to a chain of compensated layers, from in-memory cache up through full operating system. Section 7 examines the limit case, where the chain collapses to a domain artifact deliverable as an appliance. Section 8 examines what the lens makes visible and what it does not.

2. Related work

The genre of this paper is closer to *theory for analyzing* (Gregor, 2006) than to design theory. An analytic paper of this kind identifies a typology, derives the

indicators under which the typology reads, and presents an instantiation that demonstrates the reading produces non-trivial discrimination. The contribution is conceptual; the instantiation is the existence proof, not the substance of the claim.

Critique of DBMS-centric architecture as the dominant paradigm for production systems has accumulated in the literature without naming the construct presented here. Stonebraker and Çetintemel (2005) observe that the relational database has become a one-size-fits-all hammer applied to use-cases for which it is structurally ill-suited, and predict the proliferation of specialized stores. Kleppmann (2017) catalogs the patterns by which contemporary deployments combine caches, queues, ORMs, and search indices around a relational core, documenting their pairwise interactions and operational costs. Helland's *Life beyond Distributed Transactions* (2007) makes a parallel observation about the proliferation of coordination machinery under distributed-transaction semantics. His *Immutability Changes Everything* (2016) extends a related argument to the substrate itself: append-only representation dissolves much of that machinery by construction. The present paper names what these critiques document piecewise — the unifying property of compensation — and renders the discrimination available as a diagnostic lens.

The instantiation presented in §4 is constructed from three lines of prior work. The actor model originates with Hewitt (1973), articulating computation as message passing between isolated entities. Event sourcing as a persistence pattern is associated with Fowler (2005), which characterizes the program's history as the durable artifact rather than a snapshot of current state. Command Query Responsibility Segregation, articulated by Young (2010), separates the read and write paths so they no longer compete for the same model. Vernon (2015) brings these three together as practical patterns in *Reactive Messaging Patterns with the Actor Model*; that synthesis is the immediate ancestor of the present instantiation. The journaled actor-native system used as the existence proof here combines these threads; it does not invent them.

This paper is the sixth in a series. The series develops the construct's foundations across five preceding papers. Paper 1 introduces porosity as the representational defect that this paper's construct identifies in its compensation form. Paper 2 develops program-value separability and the externalized-parameters precondition that supports journal density as a §4.1 mechanism. Paper 3 introduces Reactions and the pragmatic partition between work-due-before-responding and deferred work, which §4.1 invokes as the journal-as-causal-substrate mechanism. Paper 4 introduces the Tell primitive and cross-actor causal continuity, used in §4.1 for the same mechanism. Paper 5 develops the journal as substrate for deployment, replication, backup, and offline operation, which §6 references as the substrate-mediated lifecycle that dissolves application-server scaffolding. Paper 7 carries the same discrimination upward from infrastructural layers to architectural roles; §9 introduces this extension.

The contribution unique to this paper is conceptual. The construct *infrastruc-*

tural symptom names a property that prior work documents in pieces but does not unify. The two diagnostic indicators — compensation and dissolution — give the property its shape. The auditor workflow of §5 guides its application without requiring adoption of any particular alternative. The ladder of §6 and the limit case of §7 extend the typology's reach. The instantiation of §4 demonstrates that the typology reads cleanly under the indicators; the demonstration is not the contribution.

3. The construct: infrastructural symptom

We state the construct before naming its two diagnostic indicators and the workflow that guides their application.

An infrastructural symptom is a layer of infrastructure that exists to compensate for a structural deficiency in the underlying persistence model rather than for a structural requirement of the problem being solved.

The construct operates at the level of purpose, not at the level of product. A given physical layer in a deployment may serve multiple purposes; the construct discriminates among those purposes one at a time. The unit of analysis is the use-case, not the layer.

A layer reads as an infrastructural symptom for a given use-case when both of two diagnostic indicators are positive.

The compensation indicator. The layer's purpose for that use-case is traceable to a specific deficiency of the underlying persistence model. The deficiency must be identifiable: not "we use this layer because it is fast," but "we use this layer because the persistence model exhibits property P, and this layer compensates for P." When the compensation indicator is positive, asking what would happen if P were not present? yields a non-trivial answer.

The dissolution indicator. When that deficiency is removed — by adopting a persistence model in which P does not arise — the layer loses justification for that use-case. The deficiency's absence is sufficient to make the layer redundant for that purpose; nothing else about the problem domain, workload, or operational context needs to change.

The dissolution indicator requires that M' — the counterfactual model in which P does not arise — be *constructible*: a persistence model with operationally defined semantics, viable under the workload, hardware, and programming-model constraints of the use-case under examination. A counterfactual whose viability is unspecified — an imagined absence of P without a model that exhibits the absence in operational form — does not count as a model in which D does not arise; it only restates the wish that D were absent. The lens does not admit such counterfactuals. The instantiation of §4 is offered precisely as one such constructible M', sufficient to make the dissolution indicator non-vacuous for the seven canonical categories.

Both indicators must be positive. Compensation without dissolution describes a genuine patch whose value survives model evolution. Dissolution without compensation is not constructible: a layer cannot dissolve under a model change unless its purpose was tied to the model in the first place.

The construct is not a universal accusation against infrastructure. Many layers in production systems exist as structural requirements — they serve purposes rooted in the problem domain, independent of any persistence model. Three categories are typical:

- **External-system integration.** Layers mediating communication with systems outside the boundary — payment gateways, partner APIs, sensor networks — compensate for no model deficiency. The integration problem remains under any persistence model.
- **Cryptographic and protocol enforcement.** TLS termination, request signing, OAuth verification address security or protocol requirements rooted in the problem domain. No persistence-model change makes encryption optional.
- **Genuinely expensive computation.** Analytics engines, machine-learning inference services, video encoders exist because the computation itself is intrinsically expensive. That cost does not change under a different persistence model.

The boundary between symptom and structural requirement is not a property of the layer as a product but of the purpose for which it is used. The same caching system, deployed to absorb read-vs-write contention at the persistence layer, instantiates an infrastructural symptom. Deployed instead to memoize a deliberately expensive computation, it instantiates a structural requirement. The reading is produced per use-case.

Seven categories of layer are encountered regularly in production deployments and admit reading under the two indicators. The third column describes, in model-property terms rather than system terms, what a persistence model under which both indicators read positively would offer instead.

These seven are not a privileged set but an entrenched one: the layers most consistently assumed by default in conventional production stacks, taught as foundational, and budgeted as standing operational concerns. Other commonly-deployed layers — logging frameworks, schedulers, service discovery, secret managers, feature-flag servers — admit reading under the same workflow of §5 and produce more varied results. Some dissolve (in-process logging, internal scheduling and discovery); some are structural (secret managers, whose purpose is rooted in access control rather than persistence; the A/B-testing facet of feature-flag servers, rooted in product methodology); some split per use-case (logging at internal vs. external boundaries; service discovery at internal vs. external). The lens discriminates; Table 1 catalogs the architectural baseline against which the discrimination is most consequential.

Table 1. Diagnostic reading: seven canonical categories of compensating in-

frastructure under the two indicators of §3.

Layer	Underlying defect compensated	What a model with both indicators positive offers instead
In-memory cache	Reads contend with writes at the persistence layer; reconstructing state from storage is expensive	State local to the unit of computation; no contention between read and write paths
Object-relational mapper	Domain objects do not align with relational rows; mapping is opaque and lossy	Domain objects are the unit of persistence; no translation layer
Message queue used as glue	Components cannot directly invoke deferred work without losing causal record	Deferred work is journal-recorded and replayable; queues are not the substrate of causality
Distributed lock	Multiple writers contend for the same logical entity across processes	A single point of authority per logical entity; serialization is intrinsic, not externalized
Application server framework	The persistence model leaks into application code; lifecycle plumbing must be added	The domain is the application; lifecycle is a property of the substrate, not added scaffolding
Orchestration cluster as coordination substrate	Stateful coordination across instances requires external choreography	State has location; coordination is between actors, not between containers
Full operating system	The runtime, the framework, and the application require disparate substrates	A minimal substrate suffices to run the domain artifact

The seven rows are not exhaustive. Backup tooling, service mesh, and distributed-tracing frameworks admit similar analysis and are discussed in Section 6. Whether such a persistence model exists, and at what cost, is the subject of Section 4.

The construct is diagnostic, not prescriptive. It enables auditing existing stacks under the two indicators; it does not specify how to build a model in which

infrastructural symptoms do not arise. Construction is the work of Section 4, which exhibits one such model, and of Section 5, which specifies the auditor workflow that produced the diagnostic table.

The value of the typology is diagnostic. A practitioner who does not adopt the instantiation of Section 4 can nevertheless use it to read which layers in their stack are tied to their persistence model and which are tied to their problem domain. That visibility is independent of the choice to act on it.

4. Instantiation

4.0 Genealogy

Puppeteer's earliest layer dates to 2005, when a persistence library internally called *autopersistencia* adopted a single design rule: the classes representing the domain would carry no persistence concerns. Persistence would be discovered by reflection over the class hierarchy; storage details lived elsewhere. The rule was modest in scope and pragmatic in motivation. Its consequence was structural: the domain became a clean substrate, unobstructed by the technical aspects that conventional architectures embed inside it.

At no point in this evolution was there an intention to eliminate caches, queues, locks, or application scaffolding; those layers were assumed to belong in any serious system. Their later absence was not a design objective but an empirical outcome. This distinction matters for the argument of this paper: the construct is not dissolved here by architectural ideology but as a byproduct of unrelated design constraints that happened to satisfy the indicators derived in §3.

The natural follow-up question was: if domain classes carry no persistence concerns, where does state live? The answer that emerged across the following years became the framework's first generation. State lives in a journal, and the journal records the program itself — not serialized events, not row deltas, but the DSL script that produced the state, replayable in order. The narrative of execution, not the photograph of memory, became the unit of persistence — a property whose absence, in the terms of §3, is the deficiency that caches, ORMs, and queue-glue exist to compensate for in DBMS-centric architectures.

The framework's second generation added four properties, each developed in preceding papers of this series. Parameters were externalized, allowing programs to be compiled, cached, and recorded as references rather than literal scripts (Paper 2). Reactions were introduced as the developer-controlled partition between work due before responding and work deferred to placement (Paper 3). The Tell primitive established cross-actor causality recorded in the journal rather than orchestrated externally (Paper 4). The journal itself became the catalog of the actor's declarative vocabulary, dissolving the lateral storage of action definitions that the framework had carried since V1.

The framework was not designed to dissolve the construct named in this paper. The construct was identified retrospectively, after the chain of architectural

decisions described above had converged on a system in which most of the canonical layers had become unnecessary. The work of this paper is, in that sense, a forensic reading of an existing artifact, not an engineering plan derived from a principle. The principle was a property of the artifact before it was a property of the literature.

4.1 The instantiation under the two indicators

Four mechanisms in the journaled actor-native model account for the dissolution of the seven canonical categories of §3. A single mechanism may dissolve more than one category, and the in-memory cache dissolves under two mechanisms because §3 records two distinct defects compensated by that layer. Each mechanism decomposes into several sub-mechanisms — properties of the model that, taken together, exhibit the umbrella behavior. Table 2 correlates each mechanism with its sub-mechanisms, the §3 defects it dissolves, and the canonical layers that become symptoms under it.

Table 2. Mechanisms in the journaled actor-native model mapped to the §3 defects they dissolve and the canonical layers that become symptoms.

Mechanism	§3 defect compensated	Canonical layer that becomes symptom
Locality • privacy of state • actor-as-serializer • placement	Reads contend with writes at the persistence layerMultiple writers contend for the same logical entity	In-memory cache (contention defect)Distributed lock
Journal density • journal-as-program (homoiconic) • replay-as-reconstruction • domain-class-as-unit-of-persistence	Reconstructing state from storage is expensiveDomain objects do not align with relational rows	In-memory cache (reconstruction defect)Object-relational mapper
Journal as causal substrate • Reactions recorded in journal • Tell recorded in journal • causality boundary is the journal	Components cannot directly invoke deferred work without losing causal recordStateful coordination across instances requires external choreography	Message queue used as glueOrchestration cluster

Mechanism	§3 defect compensated	Canonical layer that becomes symptom
Self-contained substrate • domain artifact is the application • substrate carries lifecycle, persistence, dispatch • journal-mediated release handoff • minimal OS surface	The persistence model leaks into application codeThe runtime, framework, and application require disparate substrates	Application server frameworkFull operating system

Locality. State is private to the actor that owns it; the actor processes commands and queries serially against its own memory. There is no shared store from which concurrent reads must be isolated from concurrent writes. Two of the §3 defects vanish mechanically under this property: the read-versus-write contention that motivates in-memory caches, and the inter-process contention that motivates distributed locks. The compensation indicator is vacuous for both. Placement is the developer-controlled partition treated in Paper 3. This mechanism reads positively under the actor-native preconditions established across the preceding papers in this series: the actor's state fits in memory and is processed serially over its private storage (Papers 1 and 3), and cross-actor continuity is mediated through the journal by Reactions and Tell rather than through shared queries (Papers 3 and 4). Aggregates that exceed the in-memory precondition — extremely hot single entities, datasets larger than a single host's working set — fall outside this reading and require actor splitting or partitioning, which is a design technique distinct from the lens itself.

Journal density. The journal does not record serialized snapshots or row-by-row deltas; it records the program — the DSL script — that produced the state. Reconstruction is replay, not deserialization; replay of a compact script is cheap. The domain class is the unit of persistence, with no intermediate row to map. Both §3 defects compensated by caches and ORMs — reconstruction expense and object-row impedance — are dissolved mechanically. The mechanism is treated as compilation in Paper 2 and as homoiconic representation in Paper 1. Implementation: `Puppeteer/EventSourcing/DB/Diary.cs`. Replay cost for long-lived actors is bounded by complementary substrate primitives: Distill compacts the journal in place (`Puppeteer/EventSourcing/DB/Diary.cs:264`); elision markers and Materialize watermarks enable offload-and-trim through a materialization destination (Paper 5, §7.3). Together these primitives fill the role that Rolling Snapshots (Young, 2010) play in conventional event-sourced systems, without re-introducing snapshot semantics.

Journal as causal substrate. Pragmatic deferral, developed in Paper 3 as Reactions, records work the verb does not perform before responding as a jour-

nal entry, with the causal link to the originating event preserved. Cross-actor continuity, developed in Paper 4 as Tell, records dispatch from one actor to another in the journal as well. The journal becomes the boundary of causality. In DBMS-centric systems, causality crosses the persistence boundary and must be reconstructed with message queues used as glue and orchestration clusters used as coordination substrates; here, causality never leaves the journal. Implementation: the `reactions` field in `Puppeteer/EventSourcing/ActorHandler.cs:38`.

Self-contained substrate. The runtime is a small loader; the application is the domain artifact. Lifecycle, persistence, and dispatch are properties of the substrate, so no external process is required to host or coordinate the application artifact. Release handoff and rolling deployment, handled in DBMS-centric deployments by application server orchestration, are journal-mediated here and are the subject of Paper 5. The application server framework as a category has no equivalent role; the operating system surface is reduced to the minimum required, a limit case treated in §7.

Together, these four mechanisms account for the dissolution of the seven canonical categories of §3 mechanically, not by intent. None was added to the model with the construct of this paper in mind; each was developed for the reasons enumerated in §4.0.

4.2 The Redis case in detail

Redis occupies a distinctive position in the contemporary infrastructure landscape: a single product whose presence is taken for granted across deployments that are otherwise architecturally dissimilar. The diversity of its typical use-cases — cache, session store, pub/sub, queue, distributed lock, leaderboard, rate limiter — does not reflect the diversity of a single problem domain. It reflects the diversity of the pressures that DBMS-centric architectures present to application designers, each of which Redis conveniently absorbs in a familiar form. This makes Redis the canonical case for inspection under the construct of §3. The analysis that follows is not, however, about Redis as a product; it is about the defects of the underlying persistence model that Redis exists to absorb — defects that other compensating layers also address in variant forms, treated briefly after the canonical table.

Table 3 applies the two indicators of §3 to the most common Redis use-cases, tracing each to the specific defect it compensates and the mechanism in §4.1 that dissolves it.

Table 3. Application of the diagnostic typology to canonical Redis use-cases.

Redis use-case	§3 defect compensated	Native response in journaled actor-native
In-process cache for database results	Reads contend with writes at the persistence layer; reconstructing state from storage is expensive	Actor state is the cache (§4.1 Locality + Journal density)
Session storage	Stateless application servers cannot retain user state across requests	The user actor retains session naturally as part of its state (§4.1 Locality + Self-contained substrate)
Pub/sub among internal components	No causal record of cross-component events; coordination must be reconstructed	Reactions record deferred work in the journal with causal links preserved (§4.1 Journal as causal substrate)
Message queue for deferred work	Components cannot directly invoke deferred work without losing causal record	Reactions and Tell record causation in the journal (§4.1 Journal as causal substrate)
Distributed lock for shared entity	Multiple processes write the same logical entity across boundaries	The actor owns its entity; serialization is intrinsic, not externalized (§4.1 Locality)
Leaderboard / sorted set	Real-time aggregation against the persistence layer is expensive	Query against actor state or a follower; replay is cheap (§4.1 Locality + Journal density)

Each row reads positively under both the compensation and dissolution indicators of §3. The pub/sub row covers internal-component coordination; the external-systems variant is structural and is treated below.

Two Redis use-cases do not read positively under the dissolution indicator and are therefore structural under the typology. The first is pub/sub directed at external systems: webhooks to third-party consumers, event distribution to partners, broadcast to subscribers outside the operator's boundary. The external system does not dissolve under any persistence-model change; the distribution problem persists. The second is rate limiting applied to external API consumption: per-customer quotas, third-party API throttling, abuse mitigation against external traffic. The rate limit is a property of the external relationship, not of

the internal model. Redis, or any comparable substrate, remains the appropriate tool for both.

Other compensating layers in the DBMS-centric ecosystem exhibit variants of the same pattern. Memcached shares the cache analysis: locality and journal density make in-process caches mechanically redundant. Distributed coordinators such as ZooKeeper share the lock and coordination analysis: the actor model makes the underlying contention vacuous. Message brokers such as RabbitMQ share the queue analysis: Reactions and Tell record causation in the journal.

Event-storage products such as EventStore DB and Kafka deserve a separate observation. Both adopt a journaled posture — they record events rather than serialized snapshots, and they offer replay. But they instantiate the journal as an external infrastructure layer that the application consumes through a client API, not as the substrate of the application itself. Causality is recorded but remains external to the unit of computation; coordination between event production, event consumption, and application state still requires application-side glue. The product captures the right idea and externalizes it, leaving the application code in the compensating position. Under the construct, these products do not dissolve the symptom; they relocate it.

A second observation about these compensating layers is that they do not absorb their compensation freely. Each requires the application to carry the boilerplate of its use: cache invalidation logic, lock acquisition and release with retry and fencing semantics, serialization between domain objects and external representations, key naming and TTL discipline, consumer offset management and rebalancing handlers, schema registry coordination, idempotency keys for at-least-once delivery. The application code carries the porosity imposed by RDBMS representations (Paper 1) as its baseline; the compensating layer adds a second porosity, this time imposed by the layer itself: its own API surface, lifecycle, and consistency contracts. Domain logic interleaves with both. Under the instantiation, the compensating layer and the boilerplate that the application carries to use it are both absent. The actor's code addresses the problem domain; the substrate addresses the rest.

Redis as a product is not what the typology identifies; nor is any single compensating layer treated above. What the typology reads is the architectural pattern in which a persistence model exposes defects and successive layers — caches, brokers, coordinators, even externalized event stores — accumulate to absorb them. Replacing one such layer while retaining the model that produces the others does not change how the typology reads. The unit of change is the persistence model itself, not any individual layer.

5. Auditor workflow

The typology of §3 enables reading per use-case, but practitioners face stacks containing dozens of compensating candidates. Without an explicit workflow

for application, the typology risks remaining theoretical: invoked at the level of architecture review, but absent from the operating discipline that distinguishes one layer from the next. The workflow that follows guides the application of the two indicators of §3, producing a reading in a finite number of steps for any (layer, use-case) pair under examination. It can be applied without adopting the instantiation of §4.

```
Input:  layer L deployed for use-case U.
Output: reading in {model-dependent, structural, unclassified};
        defect D identified when model-dependent.

Step 1 - Compensation indicator
  Identify the defect D of the underlying persistence model
  for which L compensates in use-case U.
  If a defect D is identifiable, proceed to Step 2.
  If no defect is identifiable AND L's purpose is rooted in the
    problem domain (external-system integration, cryptographic
    or protocol enforcement, intrinsically expensive computation),
    return structural.
  If no defect is identifiable AND L's purpose is opaque to the
    auditor (organizational, regulatory, historical, team-skill),
    return unclassified.

Step 2 - Dissolution indicator
  Consider a counterfactual model M' in which D does not arise.
  M' must be constructible: a persistence model with operationally
    defined semantics, viable under the workload, hardware, and
    programming-model constraints of U.
  If no such M' is constructible, return unclassified.
  If L retains a role in M' for U, return structural.
  Otherwise return model-dependent with D.
```

Four notes on application. First, the unit of analysis is the pair (layer, use-case), not the layer alone. The same product may produce different readings for different uses; the workflow must be run once per use-case. Second, identifying the defect in Step 1 requires explicit attribution to a property of the persistence model. *"We use this layer because it is faster"* does not name a defect; *"we use this layer because the model serializes writes against a shared store"* does. The workflow depends on the discipline of this attribution. Third, the *unclassified* reading exists precisely so that the workflow does not silently absorb opaque or non-model-rooted layers as structural by default; an honest auditor records what the lens cannot see rather than forcing a binary outcome. Fourth, the diagnostic table of §3 and the Redis case of §4.2 are worked examples of the workflow applied to canonical and product-specific layers respectively. Each row in those tables is the reading produced by the workflow for one (layer, use-case) pair.

The workflow renders readings; it does not prescribe action. A model-dependent reading does not require that the layer be removed in the current system. It indicates that the layer's continued presence is structurally tied to retaining the persistence model that produces the defect. Practitioners may rationally retain a model-dependent layer — for legacy preservation, contractual constraint, team familiarity, migration cost — but they do so knowing that the layer carries a debt that another model would not impose. The workflow's product is visibility, not directive.

6. The ladder of compensated layers

The seven canonical categories enumerated in §3 are not a list. Read in order, they form a ladder of increasing scale: in-memory cache (per-process), object-relational mapper (within-application), message queue and distributed lock (between processes), application server framework (the application stack), orchestration cluster (across machines), and full operating system (the hosting substrate itself). The order is not arbitrary; it follows the expanding radius of the defect the layer compensates for. Each rung compensates for a deeper limitation of the model; each rung dissolves under the alternative for the same reason.

Three additional categories that §3 forward-referenced extend the ladder.

Backup and disaster-recovery tooling — point-in-time restore, write-ahead log archiving, snapshot orchestration — exists because state in DBMS-centric architectures lives in mutable rows whose history is not the primary artifact. Under the model, the journal is the backup, and rehydration is the recovery operation; both reduce to a single substrate property treated more fully in Paper 5.

The service mesh — inter-service routing, discovery, traffic shaping, mTLS, sidecar instrumentation — exists because stateless application servers, by design, do not retain coordination state internally. Under the model, much of this coordination is internal: Reactions and Tell handle deferred work and cross-actor dispatch as journal entries. The parts that cross the operator's boundary (TLS to external partners, traffic shaping for external clients) remain structural and continue to belong in a mesh or comparable substrate.

The distributed-tracing and APM stack — span propagation, trace ingestion, request-flow visualization, latency aggregation — requires discrimination per use-case, treated in Table 4 below.

Table 4. Use-case-level reading for the distributed-tracing and APM stack.

Observability use-case	Reading	Reason
Distributed tracing to reconstruct what happened inside the system	Model-dependent	The journal already records the trace; replay is deterministic
Distributed tracing across boundaries with external systems	Structural	The external system does not dissolve under any persistence-model change
APM as *"find the bug in this incident"*	Model-dependent (predominantly)	Replay localizes the bug without requiring external instrumentation
APM as continuous capacity, latency, and error-rate telemetry in production	Structural	Resource consumption is a real-world property, not a model artifact

The first and third rows read as model-dependent; the second and fourth read as structural requirements. Debug as a one-shot operation dissolves more cleanly than continuous telemetry because the journal substitutes directly for the debugger. Continuous telemetry is mixed: the internal-trace component dissolves, but the capacity-and-cross-system component remains. §8 treats this as a worked example of the typology's reading at the boundary of its applicability.

The typology's readings do not eliminate the products from a deployment. The framework does not dissolve APM and OTel; it dissolves the internal-tracing use-case while leaving APM and OTel plugged in at the points where they instantiate a structural requirement. The same observation applies to mesh products at external boundaries, and to backup tooling for legacy systems retained alongside a journaled core.

The ladder is not closed. New layers will emerge to compensate for new defects of DBMS-centric architectures — feature-flag servers, schema registries, secret stores, sidecars for auth and identity, configuration-as-a-service products. Each new layer can be audited under the workflow of §5 and read under the typology of §3. The ladder is open upward, sideways, and into the future because the underlying persistence model continues to generate compensations; the workflow walks each new rung as it appears.

7. At the limit: from datacenter footprint to a deliverable appliance

The workflow of §5 walks each rung of the ladder as it appears; §6 showed the ladder extends. The natural question is what remains when the workflow of §5

marks every rung as model-dependent. §7 names that limit.

What remains is the domain artifact, the journal that records its execution, and the minimal substrate that runs them. The substrate retains what is irreducibly needed: a runtime, journal storage, network I/O, and a scheduling loop. Not required are the cluster orchestrator, the application server framework, the object-relational mapper, the external cache, the distributed coordinator, and the general-purpose operating system in its hosting form. The domain artifact is the application; the substrate is what is required to run it; everything else has been audited and removed under the typology.

The artifact's architectural requirements are independent of scale. The same architecture sits behind a multi-datacenter cluster with cross-region replication (where the journal and Tell handle topology), behind a Kubernetes red-black rolling deployment (where the journal mediates the release handoff), behind a single server (where the artifact runs against local storage), and behind an on-premises appliance hosting an accounting system, a clinic management system, a hardware-store inventory and point-of-sale system, or a professional-services back office. The substrate's storage requirement reduces to two operations — appending entries and advancing a cursor over them — which together compose the substrate's single primitive of replay (Paper 5); both are executed by commodity flash at marginal cost. The RAM requirement scales with the domain's working state, not with the deployment topology. The difference between these shapes is operational variation over the same architecture. What bounds the appliance case is the customer's domain — its data volume, user count, workload heaviness — not anything the architecture introduces.

This is an engineering-compatibility claim, not an operational demonstration. The deployments of the framework's home site run in conventional datacenter infrastructure across eight domain installations; no appliance-class on-premises shipment is currently among them. What §7 argues is that the path to that shape is bounded by ordinary engineering parameters — runtime footprint, journal latency on flash storage, fleet management for updates — and not by an architectural obstacle that the lens has not already named. The barriers that conventional architectures encounter at the appliance limit (a DBMS that does not fit the box and its administration overhead, an application server that demands a separate tier, an orchestrator with no peer to coordinate with, a cache for a model that does not need it) are precisely the layers the workflow of §5 reads as symptoms. The model's permission for the appliance shape is the structural counterpart of those symptoms' dissolution.

This bears restating because of what it implies. Whole categories of small-and-medium-business software — accounting, clinic management, retail inventory, professional services — are today delivered on infrastructure scaled to a fraction of their actual workload, because the conventional stack obliges a datacenter footprint for a database, an application server, and the orchestration to keep them running. The barrier is not only to delivery but to development: building software for a single hardware store or a single clinic requires the infrastructural

expertise of running a datacenter, which is the structural reason such products often do not exist at all. The lens reads each of those layers as a symptom. What §7 names is the limit case made available by that reading: software that is shippable as hardware to the customer, hosted in a single appliance, operated by the customer or a local service provider.

At the structural limit, the architecture permits software to be delivered as a physical artifact: an appliance the customer owns, not a datacenter the vendor leases on their behalf. The barrier to that shape is not in the application or in the customer's needs — it is in the infrastructure that conventional architectures require. The barrier dissolves with the symptoms.

8. What the lens makes visible and what it does not

Sections 3 through 7 showed the typology in action: canonical layers in §3, the Redis case in §4, the workflow in §5, the extended ladder in §6, and the limit case in §7. Each of those sections assumed conditions — that the persistence model is identifiable, that use-cases are decomposable, that a counterfactual model is constructible. §8 treats what the lens makes visible and what it does not. The typology still applies; the readings become coarser and the translation to action grows more complicated.

The canonical boundary case is observability. Table 4 (§6) shows that the APM stack contains four use-cases with mixed readings: two model-dependent (internal distributed tracing, one-shot debug) and two structural (cross-system tracing, continuous capacity telemetry). Each reading is correctly assigned per use-case. But a single OTel deployment in a production system serves all four use-cases simultaneously without surgical separation. Removing the instrumentation that serves the internal-tracing use-case affects the instrumentation that serves cross-system tracing; the two share span propagation, collector pipelines, and storage backends. The typology reads the use-case, not the deployment. Operational reality does not admit the same separation, and the practitioner retains the deployment intact and carries the model-dependent use-cases with it.

Four boundaries describe what the lens makes visible and what it does not.

Quantification. The workflow returns a reading per use-case; it does not predict cost saved, performance gained, or operational complexity reduced. Quantification is the work of instrumentation, not of a diagnostic typology — distinct from a limitation, this is a property of the genre.

Stack prescription. A stack populated by model-dependent layers can be identified as suboptimal, but the typology does not prescribe an alternative. The instantiation of §4 demonstrates that one exists; it does not specify the unique alternative for any given system.

Adoption prediction. A model-dependent reading does not inform the difficulty of migration: organizational costs, contractual constraints, vendor lock-in,

and engineering skills carry independent weight.

Opaque models. The workflow requires identifying the defect of the underlying persistence model in Step 1 of §5. Systems whose persistence model is opaque — third-party SaaS exposing only an API, proprietary platforms without published internals — fall outside its applicability and are recorded as *unclassified* under the workflow's third reading.

Section 5's closing observation extends here: the visibility that the lens renders is independent of the action the practitioner takes. Model-dependent layers may remain in deployments for legitimate reasons — legacy preservation, contractual obligation, organizational constraint, migration cost. The typology documents the debt; it does not resolve it.

The lens reads cleanly where the persistence model is explicit and characterizable, where use-cases are decomposable, and where a counterfactual model is constructible. Where one of these conditions fails, the typology still applies but its readings grow coarser, or the workflow falls back to *unclassified*. Where it reads cleanly, what the lens surfaces is not a smaller stack but a domain that the model permits to be modeled, libraried, and evolved without being interleaved with compensating concerns. The utility of the typology is proportional to the decomposability of the system under examination.

Persistence is one axis along which representational pressure deforms a domain — the axis this paper has traced. There is at least one more. When interaction is modeled before the domain — when aggregates take the shape of the screen rather than the screen rendering the actor's output — the same compensation pattern reappears in a different register: UI components, view-models, request DTOs, and step-state objects accumulate around a domain that no longer fits its own substrate.

The lens of accidental category reads there with the same discrimination it produced for infrastructural layers and for the server-role. The actor's existing output boundary — the primitives by which it emits to its invoker, agnostic of who consumes — already factors transport out of the domain. This axis is not developed here.

9. Extension: from layers to roles

The typology of §3 reads among *layers* of infrastructure — components that occupy positions in a stack and compensate for the persistence model from those positions. Architectural *roles* — the entity that accepts authoritative writes, the master in replication, the bootstrap issuer, the orchestrator — admit the same reading under the same two indicators. A role is an accidental category when its necessity is traceable to a specific representational choice (what nodes replicate to share state) and the role loses justification when that choice is replaced.

The same lens therefore applies beyond layers. Under a substrate in which nodes replicate programs rather than data, state, or operations, the server-role does

not survive as a structural requirement. Its dissolution is not an architectural objective; it is a structural consequence of the same representational choice that dissolves the layers described in §6.

The two analyses operate at different levels of abstraction — first on layers, then on the roles that those layers exist to serve — without modifying the typology itself. The reading scales upward unchanged.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `2f31f96` (2026-05-18). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Fowler, M. (2005). Event sourcing. Retrieved from https://martinfowler.com/eaaDev/EventSourcing.html

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Helland, P. (2007). Life beyond distributed transactions: An apostate's opinion. *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 132–141.

Helland, P. (2016). Immutability changes everything. *Communications of the ACM*, 59(1), 64–70.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/01-anti-porosity.md

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/02-program-value-separability.md

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/03-reactions-and-partition.md

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/04-cross-actor-continuity.md

Rivera, A. (2026e). The journal as substrate: unifying deployment, replication, backup, and offline operation in distributed systems. *Puppeteer Papers Series*, Paper 5. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/05-substrate-operations.md

Rivera, A. (2026g). The server is not a structural requirement: identifying accidental architectural roles under journaled programs. *Puppeteer Papers Series*, Paper 7. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/07-server-as-accidental-category.md

Stonebraker, M., & Çetintemel, U. (2005). "One size fits all": An idea whose time has come and gone. *Proceedings of the 21st International Conference on Data Engineering (ICDE)*, 2–11.

Vernon, V. (2015). *Reactive messaging patterns with the actor model: Applications and integration in Scala and Akka*. Addison-Wesley.

Young, G. (2010). CQRS documents. Retrieved from https://cqrs.files.wordpress.com/2010/11/cqrs_document